

BIND: A Bidirectionally Aligned Next-Token Denoising Framework for Fast and Lightweight Deobfuscation of Harmful Web Text

Jinwoo Jung
Hanyang University
Department of Artificial Intelligence
Seoul, Republic of Korea
jinwoo0766@hanyang.ac.kr

Misuk Kim*
Hanyang University
Department of Artificial Intelligence
Department of Data Science
Seoul, Republic of Korea
misukkim@hanyang.ac.kr

Abstract

Harmful online content, including hate speech, fraud, and phishing, is increasingly disseminated in obfuscated forms designed to evade detection. This creates an urgent need for accurate and efficient real-time de-obfuscation methods to protect users and maintain trust. Existing obfuscation detection methods rely on large auto-regressive models and byte-level fallback tokenizers, which are hindered by slow inference speeds and face difficulties in handling graphemes with multiple code points and out-of-vocabulary (OOV) processing. This study proposes **Bidirectionally Aligned Next-Token Denoising (BIND)**, which integrates character-level token alignment with a novel attention technique to enable precise and efficient corrections at fixed positions. Experiments conducted on a public dataset of obfuscated harmful text demonstrate that BIND outperforms existing methods. BIND has shown strong robustness against various text-based visual, phonetic, and semantic perturbations, proving particularly resilient against emojis and other OOV elements. This research highlights how a task-specific small language model can outperform larger ones, offering a practical solution for real-time harmful content mitigation and contributing to the development of a safer and more responsible web.

CCS Concepts

• Computing methodologies → Natural language processing.

Keywords

Harmful Content Mitigation, Text Deobfuscation, Small Language Models, Decoder-as-Encoder Transformation, Web Safety and Trust

ACM Reference Format:

Jinwoo Jung and Misuk Kim. 2026. BIND: A Bidirectionally Aligned Next-Token Denoising Framework for Fast and Lightweight Deobfuscation of Harmful Web Text. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792596>

Resource Availability:

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792596>

Table 1: Examples of obfuscated text by type

Type	Original Text	Obfuscated Text
Emoji substitution	I will bomb you	I will 🇸🇨🇧 you
Visual similarity	se nd t h e asset	\$3 nd 7 h 3 @\$37
Phonetic similarity	p ie ce of work	p ea ce of work
Swapping	st up id idea	st pu id idea

The source code and related artifacts of this paper are publicly available at <https://doi.org/10.5281/zenodo.17234328>. The source code repository is available at <https://github.com/jjw0766/BIND>.

1 Introduction

On online platforms, the widespread dissemination of hate speech and illegal content often involves deliberate text obfuscation, where users intentionally modify linguistic forms to bypass automated moderation systems [3, 13, 37]. Such manipulations not only delay the identification of policy violations and degrade user experience but also accelerate the circulation of harmful information, thereby amplifying social risks [26, 36]. Moreover, numerous empirical studies have demonstrated that these obfuscation strategies substantially undermine the accuracy of existing detection models [11, 31, 47]. Consequently, text de-obfuscation has emerged as a critical component for maintaining real-time platform safety and reinforcing the reliability of the web ecosystem [40, 42]. Early research predominantly relied on statistical correction models [20] and later evolved into Transformer-based encoder [9] and encoder-decoder frameworks [9, 35]. Recently, however, the paradigm has shifted toward decoder-based large language models (LLMs) that leverage extensive linguistic priors [4, 44]. In real-world web contexts, obfuscation frequently occurs at the character level, such as through visually or phonetically similar substitutions, emoji-based cues, or character shuffling [55]. Table 1 illustrates representative examples. These modifications yield out-of-vocabulary (OOV) forms absent from standard model vocabularies, becoming a major source of degraded detection performance [7, 17, 24].

Existing character-level de-obfuscation methods generally fall into two broad categories, decoder-based and encoder-based approaches, each with inherent trade-offs. Decoder-based LLMs exhibit strong zero- or few-shot capabilities due to their rich pretraining, yet their autoregressive decoding mechanism imposes sequential dependencies that hinder alignment between input and output

lengths [27]. This limitation often necessitates post-processing modules [5, 58]. Compounding this issue, the byte-level fallback behavior of common tokenizers [12, 39, 45] fragments single characters into multiple byte tokens during OOV handling, further disrupting length consistency and precision.

In contrast, encoder-based architectures excel in fixed-length, position-aligned correction, enabling faster inference with identical input–output lengths. However, their reliance on WordPiece tokenization [41, 49], which replaces unseen words with [UNK], introduces information loss. The limited exploration of high-performance encoders in recent literature [32, 59] further constrains progress in this domain. These insights motivate a unified approach that integrates the complementary strengths of encoder and decoder architectures.

To address this gap, we introduce **Bidirectionally Aligned Next-Token Denoising (BIND)**, a framework for precise and efficient text restoration. BIND performs character-level realignment of byte-segmented tokens to preserve length consistency and applies bidirectional denoising rather than unidirectional decoding. This hybrid design combines the linguistic depth of decoders with the alignment stability of encoders, achieving both high accuracy and low latency. Furthermore, BIND employs a hierarchical two-stage pipeline consisting of a character-level detector and a token-level corrector: the detector locates obfuscated characters, and the corrector restores contextually appropriate tokens. Unlike conventional generation-based LLMs, this architecture ensures stable decoding while maintaining efficiency. Comprehensive experiments on English and Chinese obfuscation benchmarks [24, 55] confirm the robustness and effectiveness of the proposed framework under realistic online conditions.

The contributions of this study are as follows:

- We identify the autoregressive nature of decoder models and the byte-level fallback mechanism as key bottlenecks to real-time, character-level decoding of obfuscated web text.
- We show that replacing generative decoding with a detection–correction framework enables a small language model (sLM) to surpass larger LLMs on this task.
- We propose an efficient, practical solution for real-time harmful content defense.

2 Related Works

Early Work on Text Deobfuscation. Text deobfuscation stemmed from spelling correction research, relying on rule-based methods, statistical models [20], n-gram-based approaches [14], and later neural networks [25] to recover misspelled or corrupted text. However, as online communication evolved, users began intentionally altering text through obfuscation strategies [11] and adversarial perturbations [26], leading to more complex forms such as rare lexical variants and multimodal symbols (e.g., emojis) [24, 55]. These cases require contextual understanding, as traditional models struggle to restore or detect policy-violating content accurately. Consequently, there is an increasing demand for robust, context-aware models capable of fine-grained analysis in real-time online environments.

Encoder-based Text Deobfuscation. Later studies adopted Transformer based models for more precise deobfuscation using contextual information. BERT-based approaches [9, 10, 28] leveraged

bidirectional representations for spelling correction and obfuscation recovery, while hybrid methods combining encoder architectures with statistical models [16] further improved performance. Encoder–decoder frameworks [19] were also explored to handle complex sentence structures. However, most encoder-based models depend on WordPiece tokenizers [41, 49], which map rare or unseen words to [UNK], causing lexical information loss and limiting performance. Moreover, as recent research has shifted toward decoder-based large language models [32, 59], the lack of high-performance encoder-only architectures remains a key bottleneck for robust and efficient deobfuscation.

Decoder-based Text Deobfuscation. Recently, LLMs with strong linguistic transfer capabilities have popularized both zero-shot [21] and fine-tuned [46] approaches for handling complex textual obfuscation. However, the autoregressive nature of decoder-based LLMs causes input–output length mismatches, which significantly degrade performance [27]. To address this, prior studies have expanded token vocabularies [27], refined prompt instructions [51], and applied post-processing techniques [58, 60], yet these solutions demand extensive computation and large datasets while failing to fully resolve the alignment issue. Furthermore, prompt-based methods combined with rule or dictionary-based algorithms struggle to capture nuanced contextual meanings and often rely on surface-level pattern matching. These methods also suffer from slow inference, making them impractical for real-time text deobfuscation.

Byte-Level Language Modeling. To handle rare or unseen words modern LLMs use subword tokenization methods such as Byte-Pair Encoding (BPE) [39], Byte-level BPE (BBPE) [45], and Sentence-Piece [22]. These tokenizers employ a byte-level fallback mechanism that splits OOVs into smaller byte sequences, enhancing robustness against OOV issues. However, because a single character can be divided into multiple tokens, input–output sequence lengths often become inconsistent, posing a fundamental limitation for fine-grained, character-level tasks like spelling correction and text deobfuscation.

Encoderization of Decoder-based Language Models. Recent studies have sought to leverage the linguistic knowledge of pretrained decoder-based LLMs for downstream tasks such as text embedding and sequence labeling [23, 48]. LLM2VEC [2] introduced a method to convert unidirectional attention into bidirectional attention, effectively transforming a decoder into an encoder. This approach alleviated the shortage of large encoder models and achieved strong results on tasks such as named entity recognition and part-of-speech tagging. However, it still relies on a standard tokenizer with a byte-level fallback mechanism, which causes length inconsistencies during tokenization and limits its suitability for text deobfuscation tasks that require precise alignment and frequent handling of OOV words.

In summary, while prior studies have addressed text deobfuscation through various approaches, they remain limited by tokenization mismatches and difficulties in maintaining contextual consistency. For intentionally perturbed or multimodal obfuscated text, effective restoration requires not just pattern recovery but also context-aware semantic interpretation and precise alignment. To

overcome these limitations, it is essential to integrate the contextual precision of encoders, the linguistic expressiveness of decoders, and the adaptability of modern tokenization into a unified framework.

3 Problem Definition

Before introducing our proposed methodology, we first define two key challenges that arise when directly applying existing decoder-based LLMs to text deobfuscation tasks.

3.1 Autoregressive Constraints in Character-level Deobfuscation

While LLMs achieve impressive sentence-level understanding, they still struggle with fine-grained character-level recognition and restoration. Recent studies show that despite excelling in complex linguistic tasks, LLMs often yield inconsistent results in simple character-based operations [50, 56]. Text obfuscation commonly replaces characters with visually, phonetically, or semantically similar symbols while preserving sentence length [24, 55].

Building on this property, prior work [27] formulated text correction as a fixed-length restoration task. Likewise, we define text deobfuscation as a character-level process that reconstructs a string of the same length as the input. However, decoder-based LLMs, due to their autoregressive generation mechanism, struggle to preserve consistent input–output lengths, creating structural challenges. As shown in Figure 1, this issue becomes more pronounced with longer sentences, where accumulated length mismatches significantly degrade performance. In contrast, encoder-based models maintain stable fixed-length restoration even with fewer parameters.

3.2 Tokenization Misalignment from Byte-level Fallback

As discussed in Section 3.1, autoregressive models inherently struggle with fixed-length restoration due to their sequential token generation. This limitation can be alleviated by introducing bidirectional attention, similar to that used in encoder architectures. For instance, LLM2VEC [2] transforms a decoder’s unidirectional attention into bidirectional attention for character-level correction. However, this approach remains constrained by the byte-level tokenization adopted in most LLMs. When encountering OOV words, these tokenizers split a single obfuscated character into multiple byte tokens without preserving semantic units (Figure 4), leading to token count mismatches and hindering precise reconstruction.

4 Methodology

To address the limitations of decoder- and encoder-based models discussed in Section 1, we propose the Bidirectionally Aligned Next-Token Denoising (BIND) framework. As shown in Figure 2, BIND enhances existing models through three key components (Sections 4.1–4.3), enabling stable and precise character-level restoration.

4.1 Token Alignment

As shown in Figure 4 (left), conventional tokenizers often produce mismatched token lengths between input and target sequences. For instance, the input ‘hi’ is split into three tokens, while the target ‘hi’

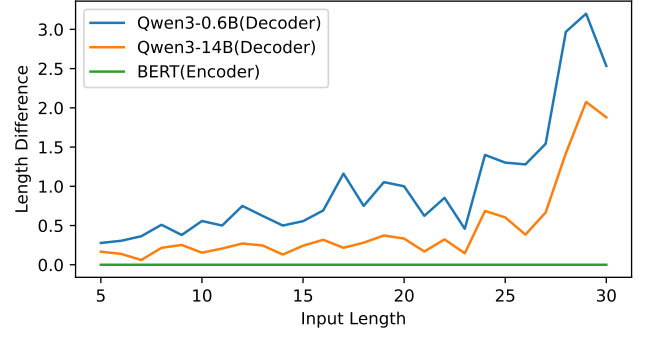


Figure 1: Length error during deobfuscation as input length increases. The y-axis shows the absolute difference between input and output sequence lengths. The encoder model preserves consistent lengths, while the decoder model exhibits growing discrepancies with longer inputs, regardless of parameter size.

yields two, creating a length discrepancy. Similar issues arise with multi-code point characters such as emojis, hindering character-level alignment and training stability. To resolve this, we apply Token Alignment, which enforces identical token lengths for X (input) and Y (target) by tokenizing both at the character level, as defined in Eq. 1.

$$X = (x_1, x_2, \dots, x_{|X|}), Y = (y_1, y_2, \dots, y_{|Y|}), \text{ where } |X| = |Y|. \quad (1)$$

In this process, OOV words and multi-codepoint graphemes (e.g., emojis) are divided into multiple tokens, with OOVs decomposed into up to four tokens via the byte-level fallback mechanism. The original tokenization function τ for a character x is defined in Eq. 2.

$$\tau(x) = (t_1, t_2, \dots, t_n), \tau(y) = (t_1, t_2, \dots, t_m), \text{ where } 1 \leq n, m \leq 4. \quad (2)$$

At this stage, the token lengths m and n of the sequences before and after deobfuscation may differ. The Token Alignment function T adjusts $\tau(x)$ and $\tau(y)$ to the same length K , as defined in Eq. 3.

$$T_K(\tau(x)) = \begin{cases} (t_1, \dots, t_{|T|}), & |T| = K \\ (t_1, \dots, t_{|T|}, \rho_{|T|+1}, \dots, \rho_K), & |T| < K \\ (t_1, \dots, t_K), & |T| > K \end{cases} \quad (3)$$

Defined in Eq. 3, tokens produced by τ are truncated if longer than K or padded with a special token ρ if shorter, ensuring equal length. The tokenization and Token Alignment function T are then applied to each character in X and Y to generate the aligned sequences $\hat{X}$ and $\hat{Y}$, as defined in Eq. 4.

$$\begin{aligned} \hat{X} &= (T_K(\tau(x_1)), T_K(\tau(x_2)), \dots, T_K(\tau(x_{|X|}))) \\ \hat{Y} &= (T_K(\tau(y_1)), T_K(\tau(y_2)), \dots, T_K(\tau(y_{|Y|}))) \end{aligned} \quad (4)$$

Consequently, both input and target sequences are decomposed into token sequences of equal length K , ensuring consistent positions for token-level training. The proposed method enforces character-level alignment and enhances deobfuscation.

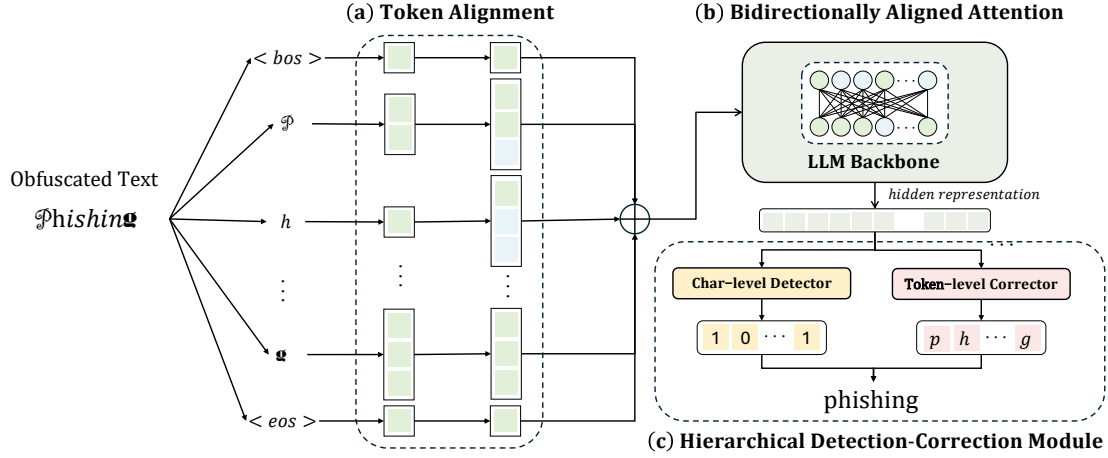


Figure 2: Overall architecture of BIND. The obfuscated text is first tokenized using a standard tokenizer, and the aligned output is concatenated through (a) Token Alignment. Then, (b) Bidirectionally Aligned Attention is applied via the LLM backbone, followed by (c) the Hierarchical Detection–Correction Module, which performs character-level detection and token-level correction.

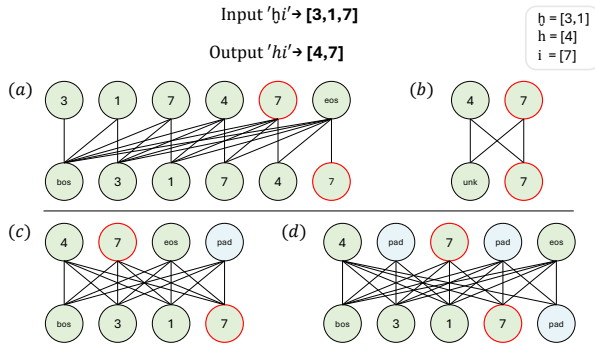


Figure 3: Comparison of different attention mechanisms: (a) causal attention in a conventional decoder, (b) bidirectional attention in an encoder, (c) applying general bidirectional attention to a decoder, and (d) applying the proposed Bidirectionally Aligned Attention to a decoder. Unlike (c), (d) preserves the alignment between the input and output tokens, as shown in the example of token 7, similar to (a).

4.2 Bidirectionally Aligned Attention

To enhance the alignment in text deobfuscation, we propose a novel attention mechanism inspired by the Masked Next-Token Prediction objective in LLM2VEC [2], addressing the limitations of byte-level fallback that occurs with rare words. Figure 3 compares different attention mechanisms under this condition. In (a) Causal Attention (standard decoder), each token i depends only on preceding tokens, predicting the next token ($i + 1$). In (b) Bidirectional Attention (encoder), each token attends to both preceding and following contexts, with rare words replaced by a single [UNK] token. In (c), direct application of bidirectional attention to a decoder leads to misaligned

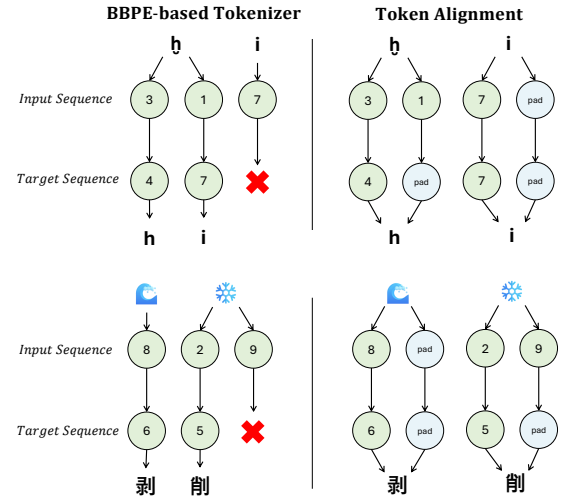


Figure 4: Visualization comparing the conventional BBPE-based tokenizer and the proposed Token Alignment. While the conventional tokenizer produces different input and output lengths, Token Alignment ensures that the input and output lengths are identical.

token correspondence. Finally, (d) illustrates our Bidirectionally Aligned Attention, which preserves the character-level mapping between input token i and output ($i + 1$), maintaining stable alignment while leveraging bidirectional context for effective denoising and prediction.

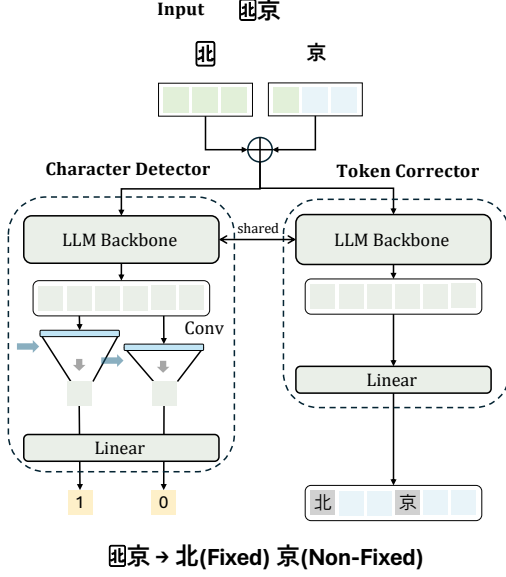


Figure 5: Architecture of the Hierarchical Detection-Correction Module. An example of deobfuscating an emoji-obfuscated input: the Character-level Detector identifies corrupted characters, and the corresponding tokens are restored by the Token-level Corrector.

4.3 Hierarchical Detection-Correction Module

The Hierarchical Detection-Correction Module (HDCM) performs character-level detection and token-level correction in a hierarchical manner. Detailed descriptions of each component are provided in the following sections.

4.3.1 Character-Level Detector. The transformed input sequence $\hat{X}$ from Section 4.1 is passed through the LLM Backbone, producing the hidden representation H_{token} (see Eq. 5).

$$H_{token} = LLMBackbone(\hat{X}) = (h_1^1, \dots, h_K^1, h_1^2, \dots, h_K^{|X|}), \quad (5)$$

where $h_j^i \in \mathbb{R}^d$, $i \in [1, |X|]$, and $j \in [1, K]$. A convolution layer with a stride of K is then applied to aggregate the decomposed tokens and extract the character-level features H_{char} as shown in Eq. 6.

$$H_{char} = Conv(H_{token}) = (\hat{h}_1, \hat{h}_2, \dots, \hat{h}_{|X|}), \text{ where } \hat{h}_i \in \mathbb{R}^d. \quad (6)$$

The extracted character-level features H_{char} are passed through a linear layer to produce the binary probability prediction $\hat{y}$, as shown in Eq. 7.

$$\hat{y}_i = \sigma(W\hat{h}_i + b), \quad \hat{y}_i \in [0, 1]. \quad (7)$$

Here, W and b denote the weight and bias of the output linear transformation. Consequently, using the predicted values, the detection loss L_{detect} can be defined as shown in Eq. 8).

$$\mathcal{L}_{detect} = \frac{1}{|X|} \sum_{i=1}^{|X|} \text{FocalLoss}(\hat{y}_i, y_i) \quad (8)$$

Since the task involves predicting a relatively small number of obfuscated characters, Focal Loss [29] is applied.

4.3.2 Token-Level Corrector. Consistent with the design of the Character-level Detector, the Token-level Corrector applies Token Alignment to the obfuscated input and performs token-wise deobfuscation. The hidden representation H_{token} obtained in Section 4.3.1 is passed through the model's output layer to perform deobfuscation, as shown in Eq. (9)

$$\hat{y}_j^i = \text{Softmax}(h_j^i \cdot W + b), \quad \hat{y}_j^i \in \mathbb{R}^{|V|}, \quad (9)$$

In this equation, W and b represent the weight and bias of the output linear transformation, and $|V|$ denotes the size of the model's vocabulary. The *Loss* is computed using the *CrossEntropy* between the predicted token y_j^i and the ground-truth token $\hat{y}_j^i$, as defined in Eq. 10.

$$\mathcal{L}_{correct} = \frac{1}{|X|} \sum_{i=1}^{|X|} \left(\frac{1}{K} \sum_{j=1}^K \text{CrossEntropy}(p_{i,j}, \hat{t}_{i,j}) \right), \quad (10)$$

$p_{i,j}$ denotes the predicted probability distribution produced by the model for the input sample x_i at the j position, and $\hat{t}_{i,j}$ represents the one-hot vector of the corresponding ground-truth token. The final training objective is defined as Eq. 11.

$$\mathcal{L} = \mathcal{L}_{correct} + \lambda \cdot \mathcal{L}_{detect} \quad (11)$$

For robust experimentation, λ is fixed to 1 without further optimization. Additional experiments regarding this setting are provided in Appendix A.1. During inference, correction is performed only on characters detected as obfuscated by the Character-level Detector.

5 Experimental Setups

5.1 Datasets

To evaluate the proposed method, we conduct our main experiments on publicly available datasets that cover diverse obfuscation patterns: ToxiBenchCN [55], which contains both Chinese and English text, and the English phishing datasets BitAbuse and BitViper [24]. Examples from each dataset are shown in Table 2. ToxiBenchCN involves multimodal obfuscation; in our main setup, we focus on the following three types:

- **Visual:** replaces characters with visually similar ones or converts simplified to traditional Chinese.
- **Phonetic:** substitutes words with pinyin initials or homophones of similar pronunciation.
- **Semantic:** shuffles character order within words or replaces words with semantically related emojis.

We split ToxiBenchCN into 80% training, 10% validation, and 10% testing sets. BitAbuse and BitViper contain English text augmented from real phishing scenarios, mainly involving visual obfuscation with similar-looking characters. Following the challenging setup of Lee et al. [24], we split the data into 1% for training, 20% for validation, and 79% for testing. To further examine the generality of our method beyond the main benchmarks, we additionally report results on an additional language (Korean) in the Appendix A.2.

Table 2: Examples of text obfuscation types in the experimental datasets. The table presents representative obfuscation patterns and examples from the ToxiBenchCN, BitAbuse, and BitViper datasets used in our experiments. Text segments highlighted in green indicate the obfuscated portions.

Dataset	Type	Category	Original Text	Obfuscated Text	Translation (in English)
ToxiBenchCN [55]	Visual	VSim	他们没有 智商	他们没有 旨商	they have no IQ
		Trad	废物才需要联合	廢物才需要联合	only t*ash needs to unite
	Phonetic	PY_Init	典型的 傻呗	典型的 sb	typical i*iot
		Homo	你 狗屁 才艺	你 够皮 才艺	your b*llshit talent
	Semantic	Shuff	你神 经了?	神你 经了?	are you c*azy?
		Emoji	死 了没有	💀 了没有	are you d*ad yet?
BitAbuse [24]	Visual	N/A	my bitc oin a dd r e ss is	my bitc òlñ à dd i e şş is	my bitcoin address is
BitViper [24]	Visual	N/A	c o mpens ate m e \$2813	c ø mpens âtê m è \$2813	compensate me \$2813

5.2 Baselines

To evaluate BIND comprehensively, we compared it with both state-of-the-art language models and traditional correction methods under different settings. For multimodal deobfuscation, baselines included the encoder-only model BERT [9], encoder-decoder models ByT5 [52] and mT5 [53], decoder-only LLMs Qwen3 [54] and Gemma3 [43], and external APIs such as GPT-4.1, o4-mini, and DeepSeek [6] (with reasoning mode). For single visual deobfuscation, we additionally included traditional methods SimCharDB [42], OCR [38], and Spell Checker [17], as well as LLM-based baselines GPT-4o-mini, BERT, Gemma3-1B, and Qwen3-0.6B.

5.3 Implementation Details

The proposed BIND framework was implemented using Qwen3-0.6B as its backbone model. Since this study aims to achieve precise correction through one-to-one mapping between input and output lengths, experiments were conducted only on tasks where the text length remains unchanged. For evaluation, the ToxiBenchCN dataset employed $chrF^{++}$ [34], which considers both character- and word-level n-grams, while the BitAbuse and BitViper datasets were evaluated using word-level accuracy [18], word-level Jaccard, and BLEU [57] scores. Further implementation details are provided in Appendix A.4, and details regarding text length can be found in Appendix A.3.

6 Experimental Results

6.1 Results on the Multimodal Deobfuscation

We first evaluated deobfuscation performance on the multimodal dataset ToxiBenchCN, with results visualized in Figure 6 and summarized in Table 3. BIND achieved the highest overall performance, attaining an average $chrF^{++}$ score of 65.65 and surpassing all baselines. It recorded 70.35 for visual obfuscation (VSim), 94.13 for traditional character transformation (Trad), and 79.37 for semantic obfuscation (Emoji), demonstrating strong restoration capability under OOV perturbations and complex character-level obfuscation. In terms of efficiency, BIND achieved an inference speed of 0.03 s/sent, about 45× faster than Qwen3-14B (1.36 s/sent) and more efficient than external API models, indicating its suitability for real-time deobfuscation.

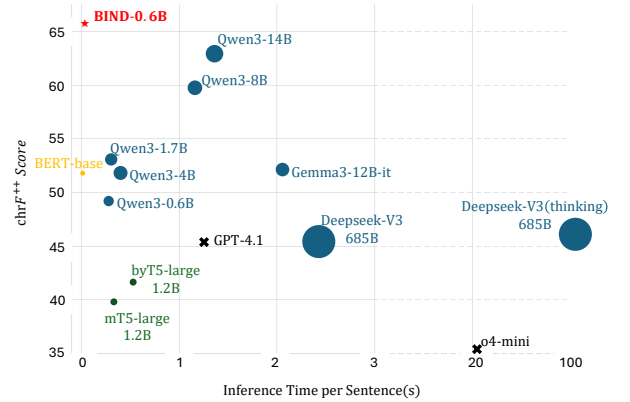


Figure 6: Visualization of harmful text deobfuscation performance on ToxiBenchCN. The x-axis shows inference time, and the y-axis shows performance ($chrF^{++}$). Point colors indicate model types (blue: decoder, green: encoder-decoder, yellow: encoder, red: BIND). Point size represents the number of parameters, and X-marked points denote commercial models with undisclosed parameter sizes.

The encoder-only model (BERT) achieved fast inference but low accuracy, while encoder-decoder models (ByT5 and mT5) struggled. Decoder-only models (Qwen3 and Gemma3) showed higher accuracy with larger parameters but slower inference, whereas external API models (GPT-4.1 and DeepSeek) exhibited both lower accuracy and slower performance, limiting their practical applicability. In contrast, BIND demonstrated high accuracy and efficiency across most obfuscation types, showing strong robustness to complex character perturbations. As shown in Figure 6, despite having only 0.6B parameters, BIND achieved superior accuracy compared to much larger models. Its task-specific architecture and training strategy enhance representation power and generalization, enabling effective deobfuscation with a compact model size.

Table 3: Overall performance comparison of the proposed BIND model and major language models on different obfuscation types in ToxiBenchCN. Both the $chrF^{++}$ score and the average inference time per sentence ($t_{avg}(s/sent)$) are reported. The best results are shown in bold, and the second-best results are underlined.

Model Type	Model	Parameters	<i>chrF</i> ⁺⁺ ↑						Average ↑	<i>t</i> _{avg} (<i>s/sent</i>) ↓
			Visual		Phonetic		Semantic			
			VSim	Trad	PY_Init	Homo	Shuff	Emoji		
Encoder-only	BERT-base-chinese	110M	50.4	76.52	44.46	45.52	44.37	49.5	51.8	0.01
Encoder-Decoder	ByT5-large	1.2B	41.42	54.54	35.5	35.98	38.04	43.76	41.54	0.53
	mT5-large	1.2B	38.28	58.89	34.15	31.4	34.34	41.08	39.69	0.33
Decoder-only	Qwen3-0.6B	0.6B	49.53	72.02	41.61	39.49	42.56	50.76	49.33	0.28
	Qwen3-1.7B	1.7B	52.6	76.72	46.23	42.86	44.72	54.57	52.95	0.3
	Qwen3-4B	4B	52.18	73.95	44.47	42.15	41.24	57.55	51.92	0.39
	Qwen3-8B	8B	59.21	85.61	48.66	50.02	<u>52.3</u>	63.45	59.88	1.16
	Qwen3-14B	14B	<u>61.88</u>	<u>91.68</u>	<u>50.14</u>	<u>51.7</u>	54.8	<u>67.7</u>	<u>62.98</u>	1.36
	Gemma3-12B-it	12B	52.23	78.01	42.69	42.11	43.59	55.06	52.28	2.06
External API	Deepseek-V3	685B	49.53	56.58	35.21	42.14	44.88	45.18	45.59	2.44
	Deepseek-V3(thinking)	685B	51.59	56.0	37.48	41.19	42.53	51.64	46.74	103.44
	GPT-4.1	Not Disclosed	48.34	57.91	35.94	40.59	43.9	46.02	45.45	1.26
	o4-mini	Not Disclosed	37.08	42.68	28.12	29.87	35.49	38.5	35.29	19.26
Proposed	BIND	0.6B	70.35	94.13	51.97	52.24	45.84	79.37	65.65	<u>0.03</u>

6.2 Results on Single Visual Deobfuscation

Table 4 presents single visual deobfuscation results on BitAbuse and BitViper. BIND outperformed most baselines, including recent and traditional methods, achieving the best word-level accuracy and BLEU scores on both datasets. Applying BIND to different backbones (Qwen3, Gemma3) yielded consistent gains (e.g., +0.06 BLEU and +0.08 Jaccard on BitViper with Qwen3-0.6B), demonstrating strong generalizability and robustness across architectures.

6.3 Qualitative Analysis

Qualitative evaluation on ToxiBenchCN emoji obfuscation reveals that BIND provides the most faithful restoration, accurately recovering toxic expressions and structure while maintaining contextual fluency. In contrast, baselines struggled: Qwen3 misinterpreted emojis literally or lacked coherence, and BERT produced semantically incorrect outputs. While DeepSeek-V3 (including reasoning mode) generated smoother sentences, it distorted meaning or lacked fidelity to the original content. Overall, BIND precisely restored both form and semantics, whereas other models often generalized or omitted critical toxic expressions.

6.4 Ablation Study

6.4.1 Out-of-Distribution Generalization Analysis. To evaluate OOD robustness, we trained models on two ToxiBenchCN categories and tested them on the remaining unseen one. As shown in Table 6, BIND demonstrated stable generalization, outperforming Qwen3-0.6B baselines in Semantic and Phonetic categories while remaining competitive in the Visual category. These findings suggest that BIND effectively captures modality-invariant representations, maintaining robust performance even under distributional shifts.

Table 4: Comparison of deobfuscation performance on single-view obfuscation datasets. Word-level accuracy, Jaccard, and BLEU scores were evaluated with and without fine-tuning. BIND consistently improved all metrics across different backbones (Qwen3 and Gemma3).

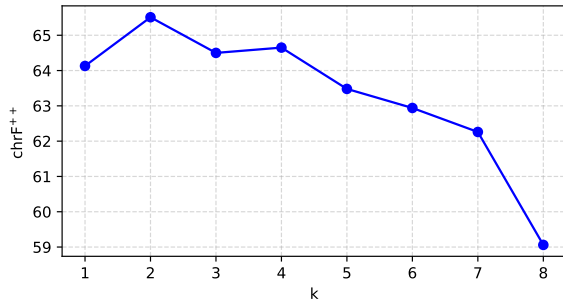
Dataset	Model	Word Level Accuracy $\uparrow$	Word Level Jaccard $\uparrow$	BLEU $\uparrow$
		w.o. Fine Tuning		
BitAbuse	SimCharDB [42]	0.35	0.49	0.78
	OCR [38]	0.34	0.48	0.78
	Spell Checker [42]	0.73	0.71	0.88
	Gpt-4o-mini	0.52	0.80	0.89
BitViper	SimCharDB [42]	0.33	0.47	0.78
	OCR [38]	0.32	0.46	0.77
	Spell Checker [42]	0.71	0.69	0.88
	Gpt-4o-mini	0.50	0.80	0.89
w. Fine Tuning				
BitAbuse	BERT [9]	0.56	0.49	0.78
	Gemma3-1B [43]	0.69	0.79	0.88
	Qwen3-0.6B [54]	0.69	0.79	0.89
	BIND(Gemma3-1B)	<u>0.84</u>	<u>0.85</u>	<u>0.95</u>
	BIND(Qwen3-0.6B)	0.91	0.91	0.97
BitViper	BERT [9]	0.53	0.44	0.76
	Gemma3-1B [43]	0.66	0.76	0.87
	Qwen3-0.6B [54]	0.62	0.77	0.83
	BIND(Gemma3-1B)	<u>0.83</u>	<u>0.84</u>	0.95
	BIND(Qwen3-0.6B)	0.85	0.86	0.95

Table 5: Comparison of deobfuscation results for harmful texts obfuscated with emojis. BIND preserved the original length and meaning while producing the most natural restorations.

Model	Text
Original Text (Ground Truth)	出她女马批手!!! 这一看就让泥哥入过了, 愿打愿挨! 看猴戏
Obfuscated Text	出她👉👉👉手!!! 这一看就让泥👉入过了, 愿打愿挨! 看👉👉
BIND (proposed)	出她女妈皮手!!! 这一看就让泥格入过了, 愿打愿挨! 看猴西
Qwen3-0.6B	出她女拳👉手!!! 这一看就让泥狗滚过, 愿打愿
Qwen3-14B	出她妈的洋盘!!! 这一看就让泥腿子入了, 愿打
BERT	出她这的枪手!!! 这一看就让泥的入国了, 愿打愿挨! 看女👉
Deepseek-V3	出她手!!! 这一看就让你入过了, 愿打愿挨! 看猴牌
Deepseek-V3(thinking)	出招! 这一看就让你卷入过了, 愿打愿挨! 看热闹

Table 6: Comparison of OOD performance between BIND and other models. Training and evaluation were conducted by excluding one obfuscation category at a time from the ToxiBenchCN dataset, showing that the proposed method consistently outperforms the baselines.

Train Dataset	Test Dataset	Model	$chrF^{++}$
Visual, Phonetic	Semantic	Qwen3-0.6B	5.45
		Qwen3-0.6B(sft)	<u>39.77</u>
		BIND (proposed)	41.14
Phonetic, Semantic	Visual	Qwen3-0.6B	10.2
		Qwen3-0.6B(sft)	51.25
		BIND (proposed)	<u>50.96</u>
Semantic, Visual	Phonetic	Qwen3-0.6B	5.51
		Qwen3-0.6B(sft)	<u>35.65</u>
		BIND (proposed)	37.16

**Figure 7: Performance variation with different k values. The model achieved the best performance at $k = 2$, while larger k values led to gradual performance degradation.**

6.4.2 Effect of k on Token Alignment. Figure 7 shows performance variations with different k values in the Token Alignment process. The hyperparameter k controls the alignment granularity between character- and token-level representations. Performance peaked at $k = 2$ and gradually declined with larger values, suggesting that two tokens per character are sufficient for semantic consistency, while excessive expansion hinders generalization.

Table 7: Performance variation with different loss configurations in BIND. The best performance was achieved when both the character-level detection loss and token-level correction loss were applied.

Loss Function	$chrF^{++} \uparrow$
$L_{correct} + L_{detect(char)}$	65.65
$L_{correct} + L_{detect(token)}$	62.65
$L_{correct}$	62.14

6.4.3 Decomposition of the Loss Function. Table 7 summarizes the performance under different loss configurations. Using only the correction loss yielded a $chrF^{++}$ score of 62.14, which slightly improved when the token-level detection loss was added. The best performance was achieved when the character-level detection loss was further incorporated, confirming that the dual detection structure across the character-token hierarchy is essential for effective deobfuscation. We further analyze the effect of weighting the correction and detection losses (i.e., the choice of λ) in Appendix A.1.

7 Conclusion

This study introduced BIND, a lightweight framework that transforms a decoder into a task-specific encoder for efficient real-time deobfuscation. By integrating bidirectional token alignment with a hierarchical detection-correction module, BIND achieves superior accuracy and efficiency across diverse backbones and datasets. As a practical solution for mitigating harmful content, future work will extend BIND to multilingual and multimodal applications for broader protection.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS2024-0040780), and Institute of Information & communications Technology Planning & Evaluation (IITP), South Korea grant funded by the Korea government (MSIT) (No. RS-2020-II201373, Artificial Intelligence Graduate School Program (Hanyang University)) and the Ministry of Education of the Republic of Korea and the National Research Foundation of Korea (NRF-2023S1A5A8081641).

References

- [1] 2025. *Obfuscated Korean Review Restoration and Generative AI Competition Dataset*. Retrieved January 23, 2026 from <https://dacon.io/en/competitions/official/236446>. Data provided for the Dacon competition "Obfuscated Korean Review Restoration and Generative AI Competition", organized on Dacon platform.
- [2] Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. arXiv:2404.05961 [cs.CL]. <https://arxiv.org/abs/2404.05961>
- [3] Meghana Moorthy Bhat, Saghar Hosseini, Ahmed Hassan Awadallah, Paul Bennett, and Weisheng Li. 2021. Say 'YES' to Positivity: Detecting Toxic Language in Workplace Communications. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Punta Cana, Dominican Republic, 2017–2029. doi:10.18653/v1/2021.findings-emnlp.173
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models Are Few-Shot Learners. In *Proceedings of the 34th Conference on Advances in Neural Information Processing Systems (NeurIPS 2020)*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.), Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 1877–1901. <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bf8ac142f64a-Paper.pdf>
- [5] Christopher Bryant, Mariano Felice, and Ted Briscoe. 2017. Automatic Annotation and Evaluation of Error Types for Grammatical Error Correction. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 793–805. doi:10.18653/v1/P17-1074
- [6] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucang Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyan Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiusi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangan Zhou, Shanhua Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shutong Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaoshan Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL]. <https://arxiv.org/abs/2412.19437>
- [7] Perry Deng, Cooper Linsky, and Matthew Wright. 2020. Weaponizing Unicodes with Deep Learning: Identifying Homoglyphs with Weakly Labeled Data. In *Proceedings of the 2020 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, Arlington, VA, USA, 1–6. doi:10.1109/ISI49825.2020.9280538
- [8] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: efficient finetuning of quantized LLMs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 441, 28 pages.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423
- [10] Bohdan Didenko and Julia Shaptala. 2019. Multi-headed Architecture Based on BERT for Grammatical Errors Correction. In *Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications*, Helen Yannakoudakis, Ekaterina Kochmar, Claudia Leacock, Nitin Madnani, Ildikó Pilán, and Torsten Zesch (Eds.). Association for Computational Linguistics, Florence, Italy, 246–251. doi:10.18653/v1/W19-4426
- [11] Steffen Eger, Gözde Gül Şahin, Andreas Rücklé, Ji-Üng Lee, Claudia Schulz, Mohsen Mesgar, Krishnakant Swarnkar, Edwin Simpson, and Iryna Gurevych. 2019. Text Processing Like Humans Do: Visually Attacking and Shielding NLP Systems. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 1634–1647. doi:10.18653/v1/N19-1165
- [12] Philip Gage. 1994. A new algorithm for data compression. *C Users J*, 12, 2 (Feb. 1994), 23–38.
- [13] Ine Gevers, Ilia Markov, and Walter Daelemans. 2022. Linguistic Analysis of Toxic Language on Social Media. *Computational Linguistics in the Netherlands Journal* 12 (Dec. 2022), 33–48. <https://www.clinjournal.org/clinj/article/view/146>
- [14] Prabhakar Gupta. 2020. A Context-Sensitive Real-Time Spell Checker with Language Adaptability. In *Proceedings of the 2020 IEEE 14th International Conference on Semantic Computing (ICSC)*. IEEE, San Diego, CA, USA, 116–122. doi:10.1109/ICSC.2020.00023
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=nZvKeeFYf9>
- [16] Yifei Hu, Xiaonan Jing, Youlim Ko, and Julia Taylor Rayz. 2020. Misspelling Correction with Pre-trained Contextual Language Model. In *2020 IEEE 19th International Conference on Cognitive Informatics & Cognitive Computing (ICCI'CC)*. IEEE Computer Society, Los Alamitos, CA, USA, 144–149. doi:10.1109/ICCIC50026.2020.9450253
- [17] Niddal H. Imam, Vassilios G. Vassilakis, and Dimitris Kolovos. 2022. OCR post-correction for detecting adversarial text images. *Journal of Information Security and Applications* 66 (2022), 103170. doi:10.1016/j.jisa.2022.103170
- [18] Niddal H. Imam, Vassilios G. Vassilakis, and Dimitris Kolovos. 2022. OCR post-correction for detecting adversarial text images. *Journal of Information Security and Applications* 66 (2022), 103170. doi:10.1016/j.jisa.2022.103170
- [19] Yannik Keller, Jan Mackensen, and Steffen Eger. 2021. BERT-Defense: A Probabilistic Model Based on BERT to Combat Cognitively Inspired Orthographic Adversarial Attacks. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 1616–1629. doi:10.18653/v1/2021.findings-acl.141
- [20] Mark D. Kernighan, Kenneth W. Church, and William A. Gale. 1990. A spelling correction program based on a noisy channel model. In *Proceedings of the 13th Conference on Computational Linguistics - Volume 2 (Helsinki, Finland) (COLING '90)*. Association for Computational Linguistics, USA, 205–210. doi:10.3115/979739.979795
- [21] Md Tawkat Islam Khondaker, Muhammad Abdul-Mageed, and Laks V. S. Lakshmanan. 2024. DetoxLLM: A Framework for Detoxification with Explanations. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 19112–19139. doi:10.18653/v1/2024.emnlp-main.1066
- [22] Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Eduardo Blanco and Wei Lu (Eds.). Association for Computational Linguistics, Brussels, Belgium, 66–71. doi:10.18653/v1/D18-2012
- [23] Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. 2025. NV-Embed: Improved Techniques for Training LLMs as Generalist Embedding Models. In *The Thirteenth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=lgslsDsDre>
- [24] Hanyong Lee, Chaelyn Lee, Jongjae Lee, and Jaesung Lee. 2025. BitAbuse: A Dataset of Visually Perturbed Texts for Defending Phishing Attacks. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 4367–4384. doi:10.18653/v1/2025.findings-naacl.247
- [25] Hao Li, Yang Wang, Xinyu Liu, Zhichao Sheng, and Si Wei. 2018. Spelling Error Correction Using a Nested RNN Model and Pseudo Training Data. arXiv:1811.00238 [cs.CL]. <https://arxiv.org/abs/1811.00238>

- [26] Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. 2019. TextBugger: Generating Adversarial Text Against Real-world Applications. In *Proceedings 2019 Network and Distributed System Security Symposium (NDSS 2019)*. Internet Society. doi:10.14722/ndss.2019.23138
- [27] Kunting Li, Yong Hu, Liang He, Fandong Meng, and Jie Zhou. 2024. C-LLM: Learn to Check Chinese Spelling Errors Character by Character. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 5944–5957. doi:10.18653/v1/2024.emnlp-main.340
- [28] Xiangci Li, Hairong Liu, and Liang Huang. 2020. Context-aware Stand-alone Neural Spelling Correction. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 407–414. doi:10.18653/v1/2020.findings-emnlp.37
- [29] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2020. Focal Loss for Dense Object Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 2 (2020), 318–327. doi:10.1109/TPAMI.2018.2858826
- [30] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Bkg6RiCqY7>
- [31] Dominik Macko, Robert Moro, Adaku Uchendu, Ivan Srba, Jason S Lucas, Michiharu Yamashita, Nafis Irtiza Tripto, Dongwon Lee, Jakub Simko, and Maria Bielikova. 2024. Authorship Obfuscation in Multilingual Machine-Generated Text Detection. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 6348–6368. doi:10.18653/v1/2024.findings-emnlp.369
- [32] Shervin Minaee, Tomáš Mikolov, Narjes Nikzad, Meysam Asgari Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large Language Models: A Survey. arXiv:2402.06196 [cs.CL] <https://arxiv.org/abs/2402.06196>
- [33] Sungjoon Park, Jihyung Moon, Sungdong Kim, Won Ik Cho, Ji Yoon Han, Jangwon Park, Chisung Song, Junseong Kim, Youngsook Song, Taehwan Oh, Joohong Lee, Juhyun Oh, Sungwon Lyu, Younghoon Jeong, Inkwon Lee, Sangwoo Seo, Dongjun Lee, Hyunwoo Kim, Myeonghwa Lee, Seongbo Jang, Seungwon Do, Sunkyoung Kim, Kyungtae Lim, Jongwon Lee, Kyumin Park, Jamin Shin, Seonghyun Kim, Lucy Park, Lucy Park, Alice Oh, Jung-Woo Ha (NAVER AI Lab), Kyunghyun Cho, and Kyunghyun Cho. 2021. KLUE: Korean Language Understanding Evaluation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/98dce83da57b0395e163467c9dae521b-Paper-round2.pdf
- [34] Maja Popović. 2017. chrF++: words helping character n-grams. In *Proceedings of the second Conference on Machine Translation*, Ondřej Bojar, Christian Buck, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, and Julia Kreutzer (Eds.). Association for Computational Linguistics, Copenhagen, Denmark, 612–618. doi:10.18653/v1/W17-4770
- [35] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. <http://jmlr.org/papers/v21/20-074.html>
- [36] Wei Ren and Yaping Guo. 2024. Translanguaging in self-praise on Chinese social media. *Applied Linguistics Review* 15, 1 (2024), 355–376. doi:10.1515/applrev-2021-0169
- [37] Fatima Salahdine and Naima Kaabouch. 2019. Social Engineering Attacks: A Survey. *Future Internet* 11, 4 (2019). doi:10.3390/fi11040089
- [38] Yuta Sawabe, Daiki Chiba, Mitsuaki Akiyama, and Shigeki Goto. 2019. Detection Method of Homograph Internationalized Domain Names with OCR. *Journal of Information Processing* 27 (2019), 536–544. doi:10.2197/ipsjip.27.536
- [39] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Katrin Erk and Noah A. Smith (Eds.). Association for Computational Linguistics, Berlin, Germany, 1715–1725. doi:10.18653/v1/P16-1162
- [40] Dev Seth, Rickard Stureborg, Danish Pruthi, and Bhuwan Dhingra. 2023. Learning the Legibility of Visual Text Perturbations. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, Andreas Vlachos and Isabelle Augenstein (Eds.). Association for Computational Linguistics, Dubrovnik, Croatia, 3260–3273. doi:10.18653/v1/2023.eacl-main.238
- [41] Xinying Song, Alex Sclianu, Yang Song, Dave Dopson, and Denny Zhou. 2021. Fast WordPiece Tokenization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2089–2103. doi:10.18653/v1/2021.emnlp-main.160
- [42] Hiroaki Suzuki, Daiki Chiba, Yoshiro Yoneya, Tatsuya Mori, and Shigeki Goto. 2019. ShamFinder: An Automated Framework for Detecting IDN Homographs. In *Proceedings of the Internet Measurement Conference (Amsterdam, Netherlands) (IMC '19)*. Association for Computing Machinery, New York, NY, USA, 449–462. doi:10.1145/3355369.3355587
- [43] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesh Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Pettrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, C. J. Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szepkator, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju-Yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Wilioughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Pöder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] <https://arxiv.org/abs/2503.19786>
- [44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] <https://arxiv.org/abs/2302.13971>
- [45] Changhan Wang, Kyunghyun Cho, and Jiatao Gu. 2020. Neural Machine Translation by Byte-Level Subwords. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 05 (Apr. 2020), 9154–9160. doi:10.1609/aaai.v34i05.6451
- [46] Mengru Wang, Ningyu Zhang, Ziwen Xu, Zekun Xi, Shumin Deng, Yunzhi Yao, Qishen Zhang, Linyi Yang, Jindong Wang, and Huajun Chen. 2024. Detoxifying Large Language Models via Knowledge Editing. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 3093–3118. doi:10.18653/v1/2024.acl-long.171
- [47] Max Wolff and Stuart Wolff. 2022. Attacking Neural Text Detectors. arXiv:2002.11768 [cs.CR] <https://arxiv.org/abs/2002.11768>
- [48] Aoqi Wu, weiqian Huang, Yifan Yang, Xufang Luo, Yuqing Fan, Chunyu Wang, Liang Hu, Xiyang Dai, Dongdong Chen, Chong Luo, and Lili Qiu. 2024. LLM2CLIP: Powerful Language Model Unlock Richer Visual Representation. In *NeurIPS 2024 Workshop: Self-Supervised Learning - Theory and Practice*. <https://openreview.net/forum?id=bNPx0aNNs>
- [49] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2016. Google's Neural Machine Translation System: Bridging the Gap Between Human and Machine Translation. arXiv:1609.08144 [cs.CL] <https://arxiv.org/abs/1609.08144>

- [50] Nan Xu and Xuezhe Ma. 2025. LLM The Genius Paradox: A Linguistic and Math Expert's Struggle with Simple Word-based Counting Problems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3344–3370. doi:10.18653/v1/2025.naacl-long.172
- [51] Zhu Xu, Zhiqiang Zhao, Zihan Zhang, Yuchi Liu, Quanwei Shen, Fei Liu, Yu Kuang, Jian He, and Conglin Liu. 2025. Enhancing Character-Level Understanding in LLMs through Token Internal Structure Learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 3839–3853. doi:10.18653/v1/2025.acl-long.194
- [52] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models. *Transactions of the Association for Computational Linguistics* 10 (2022), 291–306. doi:10.1162/tacl_a_00461
- [53] Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 483–498. doi:10.18653/v1/2021.naacl-main.41
- [54] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [55] Shujian Yang, Shiyao Cui, Chuanrui Hu, Haicheng Wang, Tianwei Zhang, Minlie Huang, Jialiang Lu, and Han Qiu. 2025. Exploring Multimodal Challenges in Toxic Chinese Detection: Taxonomy, Benchmark, and Findings. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 14382–14396. doi:10.18653/v1/2025.findings-acl.742
- [56] Xinfeng Yuan, Siyu Yuan, Yuhan Cui, Tianhe Lin, Xintao Wang, Rui Xu, Jiangjie Chen, and Deqing Yang. 2024. Evaluating Character Understanding of Large Language Models via Character Profiling from Fictional Works. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 8015–8036. doi:10.18653/v1/2024.emnlp-main.456
- [57] Guoyang Zeng, Fanchao Qi, Qianrui Zhou, Tingji Zhang, Zixian Ma, Bairu Hou, Yuan Zang, Zhiyuan Liu, and Maosong Sun. 2021. OpenAttack: An Open-source Textual Adversarial Attack Toolkit. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, Heng Ji, Jong C. Park, and Rui Xia (Eds.). Association for Computational Linguistics, Online, 363–371. doi:10.18653/v1/2021.acl-demo.43
- [58] Yue Zhang, Zhenghua Li, Zuyi Bao, Jiacheng Li, Bo Zhang, Chen Li, Fei Huang, and Min Zhang. 2022. MuCGEC: a Multi-Reference Multi-Source Evaluation Dataset for Chinese Grammatical Error Correction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, Seattle, United States, 3118–3130. doi:10.18653/v1/2022.naacl-main.227
- [59] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhiyuan Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jianyun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. arXiv:2303.18223 [cs.CL] <https://arxiv.org/abs/2303.18223>
- [60] Houquan Zhou, Bo Zhang, Zhenghua Li, Ming Yan, and Min Zhang. 2025. A Training-free LLM-based Approach to General Chinese Character Error Correction. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 13827–13852. doi:10.18653/v1/2025.acl-long.678

A Appendix

A.1 Effect of Lambda in the Loss Function

We study the weighting factor λ in our joint objective (Eq. 11), which controls the trade-off between the correction and detection losses. In particular, $\lambda = 0$ removes the detection supervision, while larger values place more emphasis on the detector.

Figure 9 shows the effect of λ on model performance (higher is better). All results are reported on the validation set and averaged over multiple random seeds.

As λ increases from 0 to 0.5, performance improves and reaches its peak around $\lambda = 0.5$. However, when λ exceeds 0.5, performance gradually declines and stabilizes near $\lambda = 1$. Notably, the performance gap between $\lambda = 0.5$ and $\lambda = 1$ is small, while fixing $\lambda = 1$ avoids additional tuning and yields stable training behavior across runs. Therefore, we use $\lambda = 1$ in all experiments for robustness and simplicity.

A.2 Korean Accommodation Review Dataset

To further examine generalization beyond the main benchmarks, we additionally evaluate on a Korean accommodation review dataset [1]. We compare BIND (Qwen3-0.6B backbone) with Qwen3-0.6B and KLUE-RoBERTa [33], using the same evaluation metric as in the main experiments. Under the default setting (i.e., the same chunk length used in the main experiments), BIND achieves the best overall performance among the three models.

A.3 Chunk-Length Sensitivity

We vary the text chunk size $L \in \{32, 64, 128, 256, 512\}$ and evaluate overall performance on the Korean review dataset A.2. As shown in Figure 8, BIND consistently performs best and remains robust to changes in L . In contrast, the Qwen3-0.6B decoder-only baseline degrades substantially as L increases, indicating higher sensitivity to longer contexts in this setting.

A.4 Implementation Details

Table 8 summarizes the fine-tuning configurations for BIND-0.6B, Qwen3-4B, and Qwen3-14B models. All experiments were conducted on NVIDIA A100 GPUs using the AdamW [30] optimizer with a learning rate of 1×10^{-4} . For each model, the effective batch size was maintained at 32 by adjusting the per-GPU batch size and gradient accumulation steps. We trained the model using chunks of length 128. During inference, we split the input into segments of the training length, run denoising on each segment, and then concatenate the outputs to obtain the final denoised text. Additional analysis on the effect of training chunk length is provided in the Appendix A.3. Training was performed for 10 epochs, and the learning rate was scheduled using a cosine annealing scheduler.

For parameter-efficient fine-tuning, the Qwen3-14B model was fine-tuned using QLoRA [8], which combines low-rank adaptation (LoRA) [15] with quantization techniques to efficiently fine-tune large language models. The LoRA rank (r) was set to 16, and the LoRA scaling factor (α) was set to 32. Relatively smaller models, BIND-0.6B and Qwen3-4B, were fine-tuned without applying quantization methods.

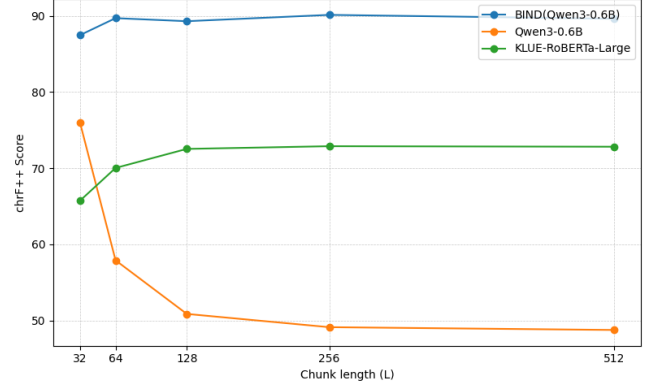


Figure 8: Overall performance on the Korean accommodation review dataset under different text chunk sizes. BIND consistently achieves the highest performance and remains robust to chunk-length variations, while the Qwen3-0.6B decoder-only baseline degrades as L increases.

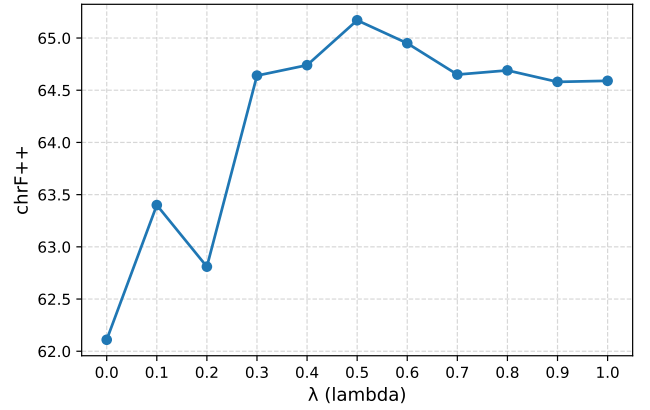


Figure 9: Performance variation with different λ values in the loss function. The model achieves the lowest performance at $\lambda = 0$ and the best performance at $\lambda = 0.5$.

Table 8: Fine-tuning configurations for BIND-0.6B, Qwen3-4B, and Qwen3-14B.

setting	BIND-0.6B	Qwen3-4B	Qwen3-14B
gpu	NVIDIA A100		
optimizer	AdamW		
learning rate	$1e-4$		
batch per gpu	32	2	32
accumulation steps	1	16	1
effective batch	32		
epochs	10		
lr scheduler	cosine annealing		
peft method	not used	not used	qlora
lora rank (r)	n/a	n/a	16
lora scale (α)	n/a	n/a	32